



UNIVERSIDAD NACIONAL DE CÓRDOBA

FACULTAD DE CIENCIAS EXACTAS, FÍSICAS Y NATURALES

CARRERA INGENIERÍA ELECTRÓNICA

**PROYECTO INTEGRADOR PARA LA OBTENCIÓN DEL
TÍTULO DE GRADO INGENIERO ELECTRÓNICO**

**“DISEÑO, DESARROLLO E IMPLEMENTACIÓN DE UN
SOFTWARE PARA LA ADQUISICIÓN Y ANÁLISIS DE ONDAS
DE IMPULSO TIPO RAYO (1,2/50 μ s).”**

Alumno

Tardón, Damián David

Director

Ing. Blasco, Marcos Javier

Co-Director

Ing Serra, Gabriel Horacio

Córdoba, República Argentina

Abril / 2026

Capítulo 1

Arquitectura del sistema

En este capítulo se presentan los fundamentos por los cuáles se adoptó Python como lenguaje de programación para desarrollar el software.

1.1 Comparativa entre Python y LabVIEW

Para desarrollar el software de control del osciloscopio y procesamiento de los datos, se analizaron dos opciones que se consideran las más adecuadas para este contexto: Python y LabVIEW. Ambas opciones ofrecen herramientas y bibliotecas para la comunicación con instrumentos de medición.

Python es un lenguaje de programación de alto nivel, conocido por su simplicidad y legibilidad. Cuenta con una amplia gama de bibliotecas y frameworks que facilitan el desarrollo de aplicaciones científicas y de ingeniería. Además, es de código abierto, gratuito y de uso libre, incluso para fines comerciales, lo que permite acceder a una gran cantidad de recursos sin tener que pagar licencias.

Por otro lado, LabVIEW, es un entorno de desarrollo gráfico utilizado principalmente en aplicaciones de ingeniería y automatización, conocido por su facilidad para crear interfaces de usuario y su capacidad para interactuar con hardware de medición. Sin embargo, es un software propietario, con licencias que comienzan en varios cientos de dólares, lo que lo convierte en una opción menos accesible.

Ambas herramientas llevan mucho tiempo en el mercado y son ampliamente utilizadas en la industria, por lo que son opciones viables para llevar a cabo este proyecto. Sin embargo, dado la diferencias de costos que tienen, Python se presenta como una opción más accesible para el contexto del laboratorio.

1.2 Bibliotecas utilizadas

Una biblioteca (o librería) en programación es un conjunto de código preescrito, funciones, clases y métodos reutilizables que permiten añadir funcionalidades específicas a las aplicaciones propias sin tener que escribir todo desde cero. Sirven para ahorrar tiempo, optimizar tareas comunes y estandarizar el desarrollo de software. Una vez elegido Python como lenguaje de programación, se procedió a seleccionar las bibliotecas necesarias. Para ello se utilizó bibliotecas estándar, integradas en Python Tabla 1.1, y de Terceros Tabla 1.2.

Librería	Función general	Rol en el sistema
datetime	Manipular fechas y horas.	Generar de marcas de tiempo para nombres de archivos.
os	Usar la funcionalidades dependientes del sistema operativo.	Leer variables de entorno y manipular rutas del sistema.
pathlib	Representar rutas para cada sistema operativo.	Manejar rutas relativas y absolutas del sistema de archivos.
platform	Identificar el sistema operativo, la versión y la arquitectura.	Identificar el sistema operativo (Windows/macOS/Linux).
re	Analizar expresiones regulares.	Validar entradas del usuario en la GUI.
struct	Realizar conversiones entre valores de Python y estructuras de C.	Desempaquetar datos del osciloscopio para convertir a int/float.
subprocess	Ejecutar comandos del sistema operativo o aplicaciones externas.	Abrir el explorador de archivos.
sys	Gestionar la comunicación entre el script y el sistema operativo.	Ejecutar y cerrar procesos.
time	Gestionar retardos, duración de procesos y relojes del sistema.	Pausar ejecución en hilos o buffers de lectura.
typing	Especificar los tipos de datos en variables, funciones y métodos.	Tipado estático y declaración de estructuras de datos.

Tabla 1.1: Librerías integradas de Python.

Librería	Función general	Rol en el sistema
fpdf2	Generar documentos PDF.	Generar documentos de calibración en formato PDF.
h5py	Almacenar datos numéricos en formato binario HDF5, y manipularlos con NumPy.	Almacenar permanentemente los datos tipeados, digitalizados y/o calculados.
matplotlib	Crear visualizaciones estáticas, animadas e interactivas.	Graficar las ondas de impulsos.
numpy	Manejar y procesar grandes volúmenes de datos.	Realizar el procesamiento digital de las señales.
openpyxl	Leer/escribir archivos Excel 2010 xlsx/xlsm/xltx/xltn.	Generar hoja de cálculo para exportar los resultados.
pandas	Analizar y manipular datos.	Estructurar datos para exportar resultados.
pyqtgraph	Visualizar datos y gráficos 2D/3D de alto rendimiento.	Graficar las señales adquiridas.
pyserial	Acceder al puerto serie.	Permitir la comunicación entre el osciloscopio y la PC.
PySide6	Crear aplicación de escritorio con interfaz gráfica de usuario (GUI).	Crear la GUI y manejar hilos/señales.
pytest	Automatizar la verificación de código, mediante pruebas unitarias.	Verificar el funcionamiento del código. Generar reporte de calibración del software.
pyvisa	Controlar instrumentos de laboratorio independientemente de la interfaz (GPIB, Serie, USB, Ethernet).	Establecer comunicación y control del osciloscopio vía protocolo VISA.
pyvisa-py	Backend para PyVISA.	Backend para PyVISA.
scipy	Incorporar funciones matemáticas complejas.	Realizar ajuste de curvas, y filtrado digital.

Tabla 1.2: Librerías externas de Python.

1.3 Definición de la arquitectura del software.**1.4 Gestión del entorno de desarrollo y control de versiones.**

Parte I

RESULTADOS Y CONCLUSIONES

ANEXOS

